

category: csharp-winforms

tags: [async-void, thread, polling, cancellation-token, application-doevents, device-driver]

severity: critical

created: 2026-07-26

updated: 2026-07-26

project-origin: DoorAlarm v3

`new Thread(async void WorkerThread)` — thread chết ngay tại `await` đầu tiên, `PollingStop()` thành vô hiệu

Tình huống gặp phải

Driver **KZE02** (bộ điều khiển cửa, giao tiếp UDP) trong **DoorAlarm v3** dùng pattern polling

kiểu cũ: **Thread** + **ManualResetEvent** để bật/tắt vòng lặp đọc trạng thái input mỗi 300ms.

Khi người dùng mở màn hình Cài đặt, app gọi **DisconnectWithDevices()** → **PollingStop()**,

đóng Cài đặt thì gọi **ConnectWithController()** → **PollingStart()**.

Triệu chứng / Lỗi

Không có exception. Triệu chứng là hành vi sai, tích lũy dần:

- Mở/đóng màn Cài đặt N lần → N vòng polling UDP cùng chạy song song tới cùng 1 thiết bị
- Thiết bị bị "spam" lệnh **GetEvent?/GetInputState?** → phản hồi chậm dần, mất event
- Thình thoảng **ObjectDisposedException** ngẫu nhiên từ **ManualResetEvent**

Nguyên nhân gốc rễ

```
public async void WorkerThread()    // ← async void
{
    while (stopEvent != null)
    {
        if (stopEvent.WaitOne(0, true)) return;
        try { await CheckNewEvent(); await CheckNewStatus(); }    // ← await đầu tiên
        finally { await Task.Delay(300); }
    }
    ...
    thread = new Thread(new ThreadStart(WorkerThread));
    thread.Start();
}
```

Thread chỉ chạy phần **đồng bộ** của **async void**. Tới **await CheckNewEvent()** đầu tiên,

method trả về → **thread vật lý kết thúc ngay**, phần còn lại của vòng lặp chạy tiếp trên

thread pool. Hệ quả dây chuyền:

1. **Running** được tính bằng **thread.Join(0) == false** → luôn **false** gần như tức thì.

2. `PollingStop()` bọc toàn bộ trong `if (this.Running)` → không bao giờ chạy, `stopEvent` không được `Set()`, vòng lặp trên thread pool cứ chạy mãi.
3. `PollingStart()` kiểm tra `if (thread == null)` — mà `Running` getter đã gọi `Free()` set `thread = null` → lần Start sau tạo thêm 1 vòng nữa, cộng dồn.
4. `Free()` gọi `stopEvent.Close()` trong khi vòng lặp còn đang `stopEvent.WaitOne(...)` → `ObjectDisposedException` ngẫu nhiên.

Thêm 1 lỗi phụ: `PollingStop()` cũ gọi `Application.DoEvents()` trong vòng chờ — bơm message loop giữa lúc đang tắt thiết bị, mở đường cho re-entrancy (user bấm nút khác ngay lúc đó).

Giải pháp

Bỏ hẳn `Thread` + `ManualResetEvent`, dùng `Task` + `CancellationTokenSource`:

```
private Task? pollingTask;
private CancellationTokenSource? pollingCts;

public bool Running => pollingTask is { IsCompleted: false };

public Task PollingStart()
{
    if (pollingTask is not null && !pollingTask.IsCompleted) return Task.CompletedTask;
    // chống chạy 2 vòng
    pollingCts?.Dispose();
    pollingCts = new CancellationTokenSource();
    pollingTask = Task.Run(() => WorkerLoopAsync(pollingCts.Token));
    return Task.CompletedTask;
}

public async Task PollingStop()
{
    var task = pollingTask;
    pollingCts?.Cancel();
    if (task is not null)
    {
        try { await task.WaitAsync(TimeSpan.FromSeconds(5)); } // CHỜ vòng cũ thoát
        thật
        catch (Exception ex) { /* log */ }
    }
    pollingTask = null; pollingCts?.Dispose(); pollingCts = null;
}

private async Task WorkerLoopAsync(CancellationToken token)
{
    while (!token.IsCancellationRequested)
    {
        try { await CheckNewEvent(); await CheckNewStatus(); }
        catch (Exception ex) { /* log, KHÔNG nuốt im lặng */ }
        try { await Task.Delay(300, token); }
        catch (OperationCanceledException) { return; }
    }
}
```

1. `async void` → `async Task (WorkerLoopAsync)`, chạy bằng `Task.Run` chứ không `new Thread`.
2. `PollingStop()` phải **await** task cũ, nếu không caller sẽ tạo vòng mới khi vòng cũ còn sống.
3. `PollingStart()` phải idempotent — kiểm tra task cũ còn chạy thì thoát sớm.
4. Bỏ `Application.DoEvents()`.

Áp dụng lại (How to reuse)

- Thấy `new Thread(...)` trở tới method có `async` → **sai ngay**, không cần đọc thêm. `Thread` chỉ dùng cho vòng lặp tuần hoàn đồng bộ (blocking I/O).
- Thấy `Running/IsAlive` dựa vào `Thread` mà body có `await` → giá trị đó vô nghĩa.
- Mọi API `Stop()` của driver thiết bị phải **await được** và phải chờ vòng lặp thoát thật; `Stop()` trả về ngay mà vòng cũ còn chạy là mằm mống "nhân đôi kết nối".
- Kiểm chứng nhanh: mở/đóng màn cấu hình 5 lần, đếm số request tới thiết bị (Wireshark hoặc log).

Chú ý / Cạm bẫy (Gotchas)

- ⚠ `async void` chỉ được phép ở **event handler WinForms**. Ở mọi chỗ khác, exception không bắt được và không await được — trong app báo động thì đây là mất cảnh báo im lặng.
- ⚠ Sau `pollingCts.Cancel()` mới được `Dispose()`; nếu `Dispose` trước khi vòng lặp thoát, `Task.Delay(delay, token)` có thể ném `ObjectDisposedException`. `Cancel` → `await` → `Dispose`.
- ⚠ `getActiveAlarmInputIds()` kiểu `lock (list) { return list; }` **không** thread-safe — caller duyệt ngoài lock sẽ ném `InvalidOperationException` khi thread khác Add/Remove. Phải `return new List<string>(list);`.
- ⚠ Đừng để `catch (Exception) { }` rỗng trong vòng polling — lỗi mạng liên tục sẽ bị giấu hoàn toàn.

Tham chiếu

- `DoorAlarmv3/Devices/KZE02.cs`, `DoorAlarmv3/Devices/KZIO0808.cs`
- Project liên quan: `DoorAlarm v3`